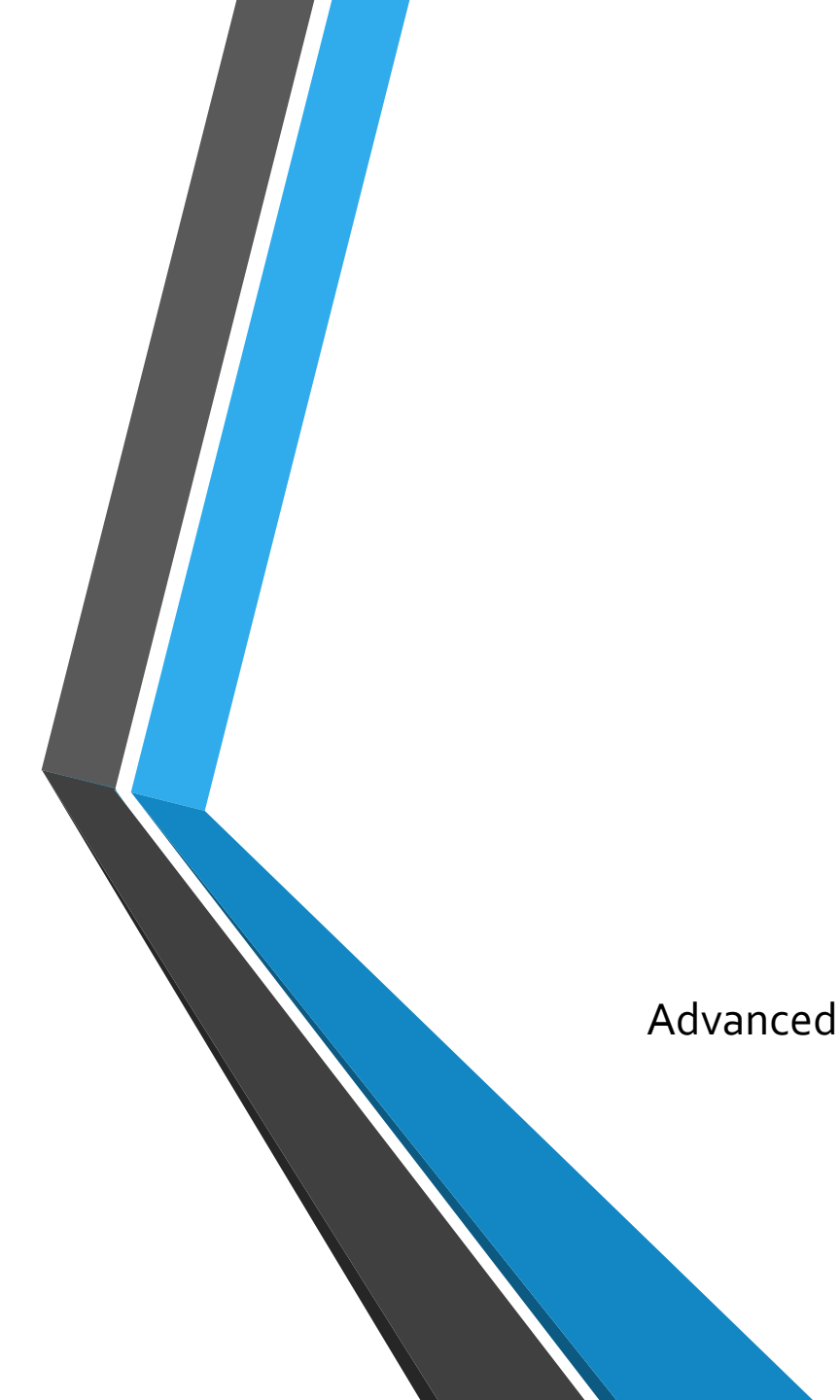




# Heaps Priority Queues Disjoint Sets

Associate Prof. Suphakant Phimoltares, Ph.D.

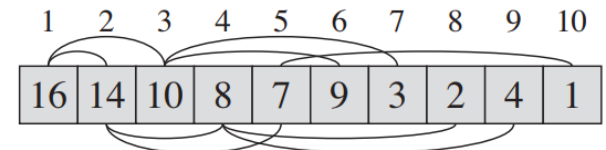
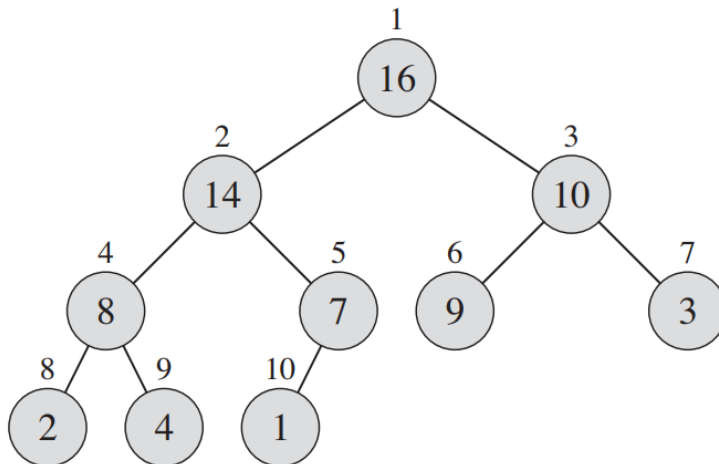
Advanced Virtual and Intelligent Computing (AVIC) Center

# Agenda

- Heaps
- Heap property
- Building a heap
- Heapsort algorithm
- Priority queues
- Disjoint sets

# Heaps

- The **(binary) heap** data structure is an array object that we can view as a nearly complete binary tree. The lowest level is filled from the left up to a point.



# Heaps

- $A$  – an array that represents a heap is an object with two attributes
- $A.length$  gives the number of elements in the array
- $A.heap-size$  represents how many elements in the heap are stored within array  $A$
- The root of the tree is  $A[1]$ .

# Heaps

- Given the index  $i$  of a node, the indices of its parent, left child, and right child can be easily computed.

PARENT( $i$ )

1    **return**  $\lfloor i/2 \rfloor$

LEFT( $i$ )

1    **return**  $2i$

RIGHT( $i$ )

1    **return**  $2i + 1$

# Heaps

- Two kinds of binary heaps: **max-heaps** and **min heaps**.
- The largest element in a max-heap is stored at the root and  $A[\text{Parent}(i)] \geq A[i]$  for every node  $i$  other than the root.
- The smallest element in a min-heap is stored at the root and  $A[\text{Parent}(i)] \leq A[i]$  for every node  $i$  other than the root.
- The height of a node is the number of edges on the longest simple downward path from the node to a leaf and the height of the heap is the height of its root.

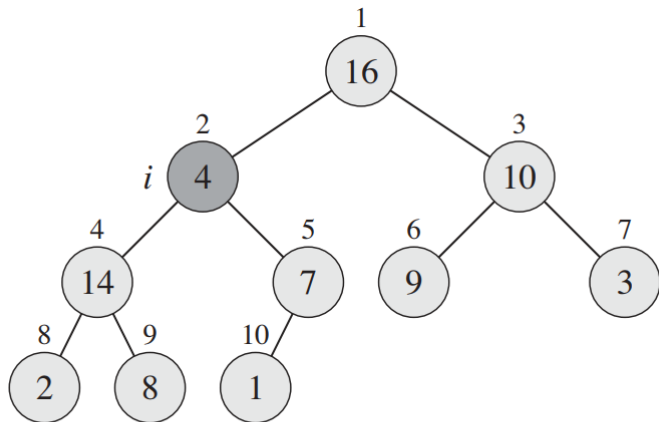
# Maintaining the heap property

MAX-HEAPIFY( $A, i$ )

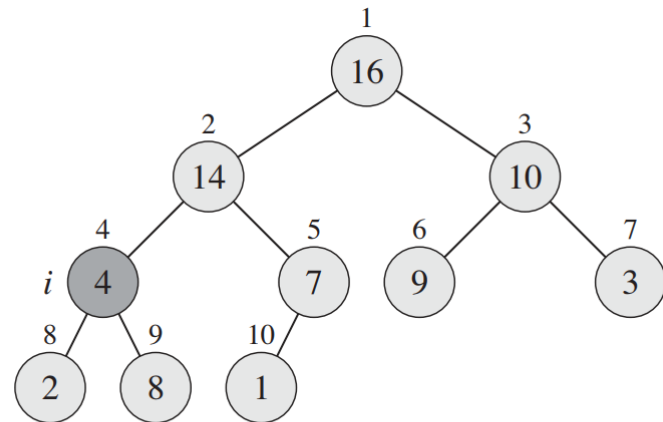
```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

Running time ?

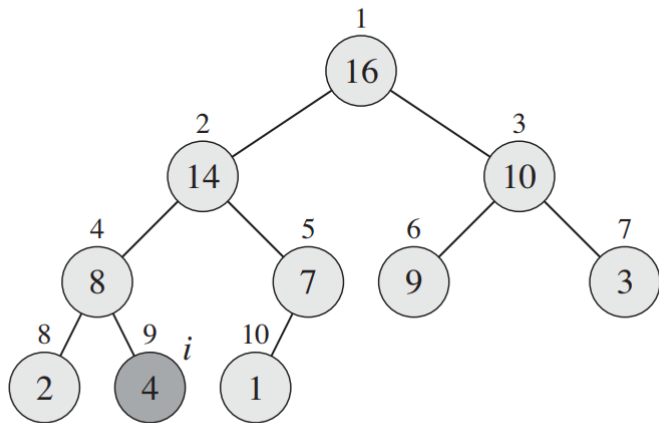
# Maintaining the heap property



(a)



(b)



(c)

# Building a heap

- Max-Heapify can be used in a bottom-up manner to convert an array  $A[1 \dots n]$ , where  $n = A.length$ , into a max-heap.

**BUILD-MAX-HEAP**( $A$ )

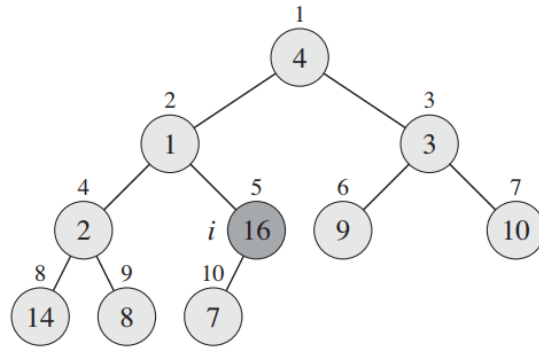
```
1   $A.heap-size = A.length$ 
2  for  $i = \lfloor A.length/2 \rfloor$  downto 1
3      MAX-HEAPIFY( $A, i$ )
```

Running time ?

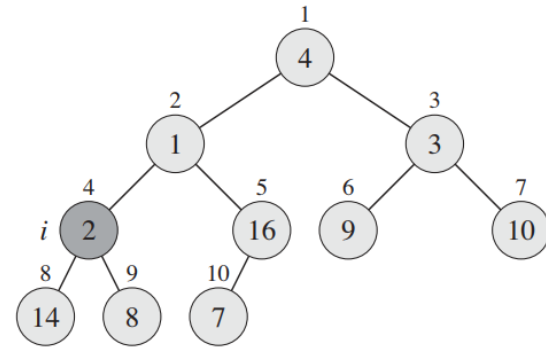
- A min-heap can be built by the procedure Build-Min-Heap with the call to Min-Heapify in line 3.

A

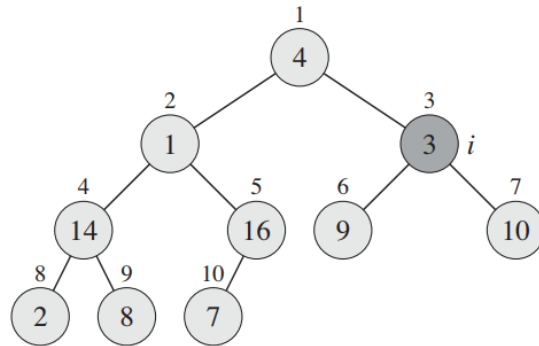
|   |   |   |   |    |   |    |    |   |   |
|---|---|---|---|----|---|----|----|---|---|
| 4 | 1 | 3 | 2 | 16 | 9 | 10 | 14 | 8 | 7 |
|---|---|---|---|----|---|----|----|---|---|



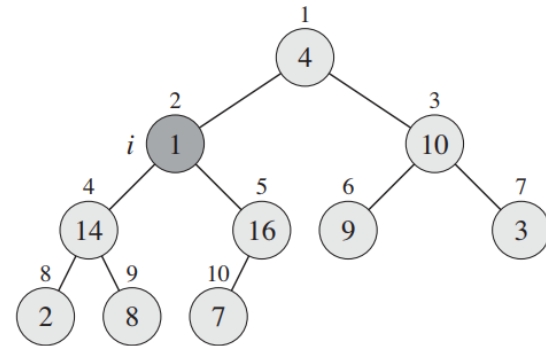
(a)



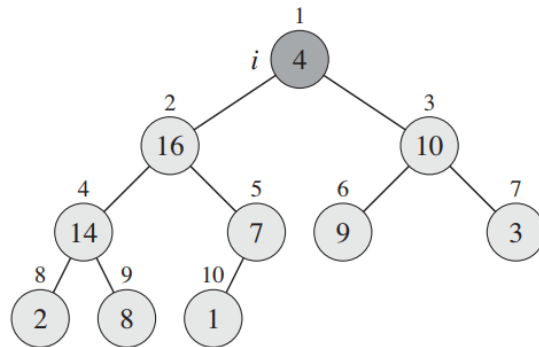
(b)



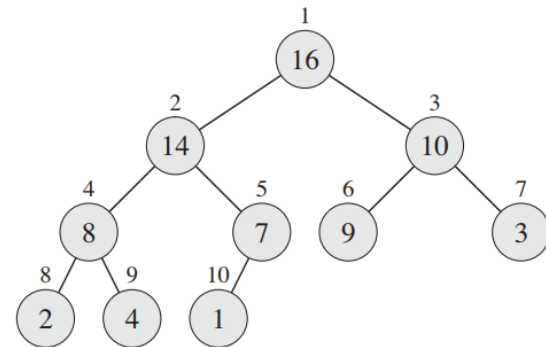
(c)



(d)



(e)



(f)

# The heapsort algorithm

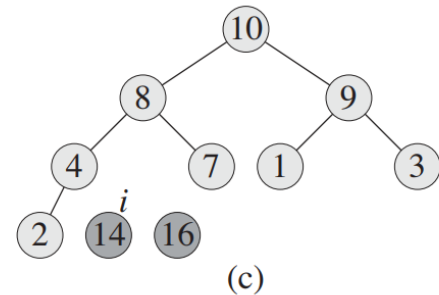
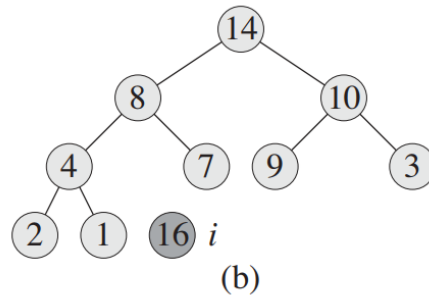
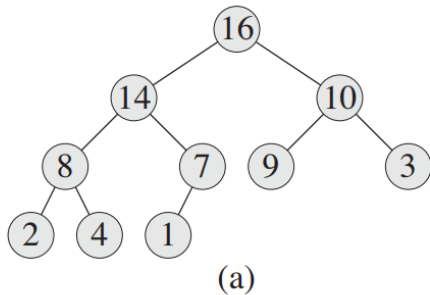
- The algorithm starts by using Build-Max-Heap to build a max-heap on the input array  $A[1 \dots n]$ , where  $n = A.length$ .
- The maximum element is stored at the root, so it can be put into the final position by exchanging it with  $A[n]$ .
- Now node  $n$  from the heap is discarded and we can do so by simply decrementing  $A.heap-size$ .
- The new root element might violate the max-heap property.
- To ensure the property, call Max-Heapify( $A, 1$ ), which leaves a max-heap in  $A[1, \dots, n - 1]$ .
- The algorithm repeats this process for the max-heap of size  $n - 1$  down to a heap of size 2.

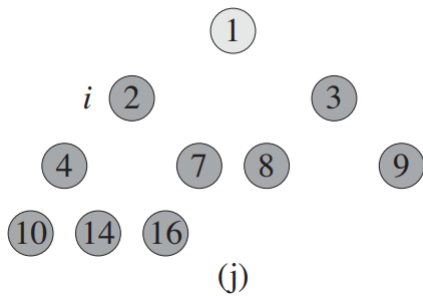
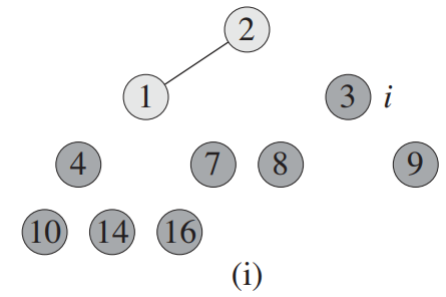
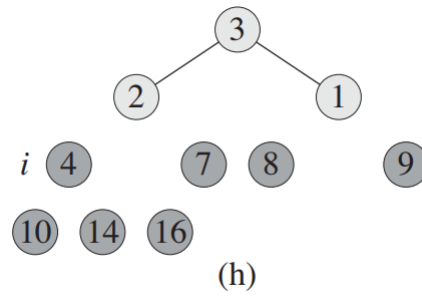
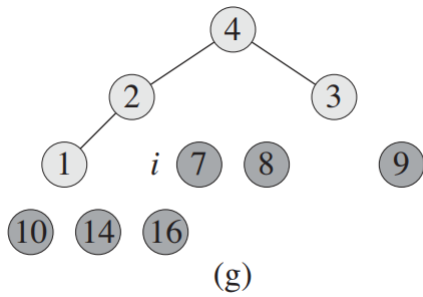
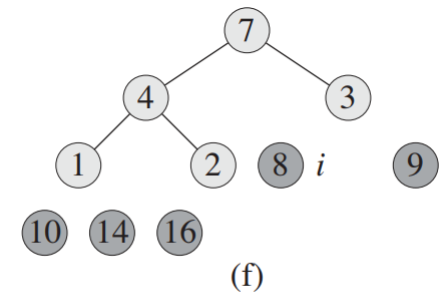
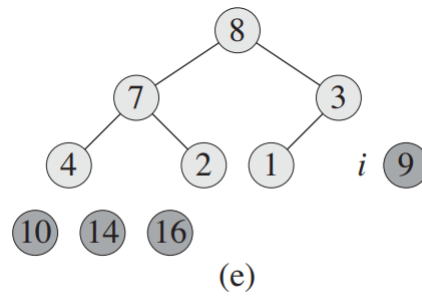
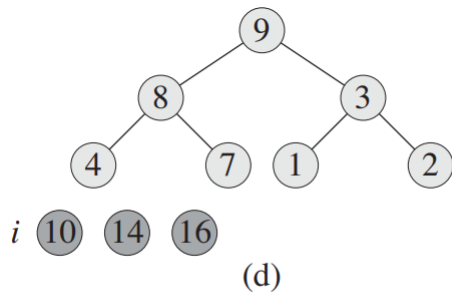
# The heapsort algorithm

HEAPSORT( $A$ )

```
1  BUILD-MAX-HEAP( $A$ )
2  for  $i = A.length$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap\text{-}size = A.heap\text{-}size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

Running time ?





A 

|   |   |   |   |   |   |   |    |    |    |
|---|---|---|---|---|---|---|----|----|----|
| 1 | 2 | 3 | 4 | 7 | 8 | 9 | 10 | 14 | 16 |
|---|---|---|---|---|---|---|----|----|----|

(k)

# Priority Queues

- A **priority queue** is a data structure for maintaining a set  $S$  of elements, each with an associated value called a **key**. A **max-priority queue** supports the following operations.
- $\text{Insert}(S, x)$  inserts the element  $x$  into the set  $S$ , which is equivalent to the operation  $S = S \cup \{x\}$ .
- $\text{Maximum}(S)$  returns the element of  $S$  with the largest key.
- $\text{Extract-Max}(S)$  removes and returns the element  $S$  with the largest key.
- $\text{Increase-Key}(S, x, k)$  increases the value of element  $x$ 's key to the new value  $k$ , which is assumed to be at least as large as  $x$ 's current key value.
- A **min-priority queue** supports the operations  $\text{Insert}$ ,  $\text{Minimum}$ ,  $\text{Extract-Min}$ , and  $\text{Decrease-Key}$ .

# Priority Queues

HEAP-MAXIMUM( $A$ )

1 **return**  $A[1]$

Running time ?

HEAP-EXTRACT-MAX( $A$ )

1 **if**  $A.heap\text{-}size < 1$

2     **error** “heap underflow”

3      $max = A[1]$

4      $A[1] = A[A.heap\text{-}size]$

5      $A.heap\text{-}size = A.heap\text{-}size - 1$

6     MAX-HEAPIFY( $A, 1$ )

7 **return**  $max$

Running time ?

# Priority Queues

HEAP-INCREASE-KEY( $A, i, key$ )

```
1  if  $key < A[i]$ 
2      error “new key is smaller than current key”
3   $A[i] = key$ 
4  while  $i > 1$  and  $A[\text{PARENT}(i)] < A[i]$ 
5      exchange  $A[i]$  with  $A[\text{PARENT}(i)]$ 
6       $i = \text{PARENT}(i)$ 
```

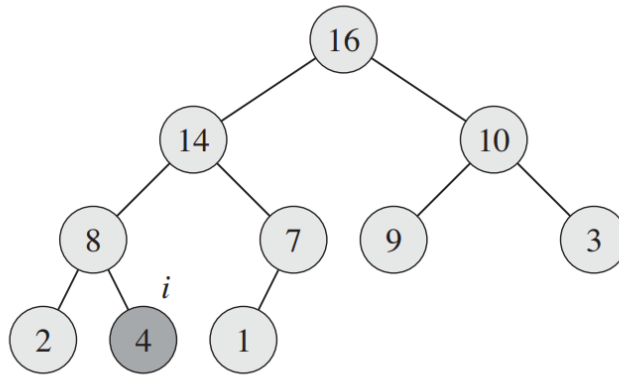
Running time ?

MAX-HEAP-INSERT( $A, key$ )

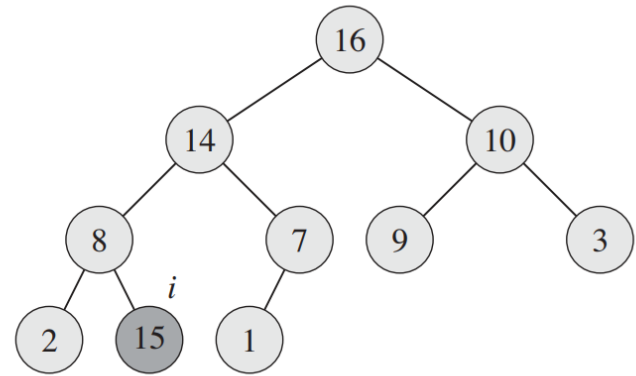
```
1   $A.\text{heap-size} = A.\text{heap-size} + 1$ 
2   $A[A.\text{heap-size}] = -\infty$ 
3  HEAP-INCREASE-KEY( $A, A.\text{heap-size}, key$ )
```

Running time ?

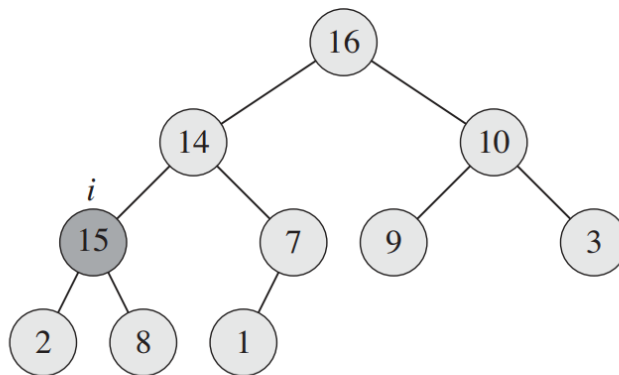
# Priority Queues



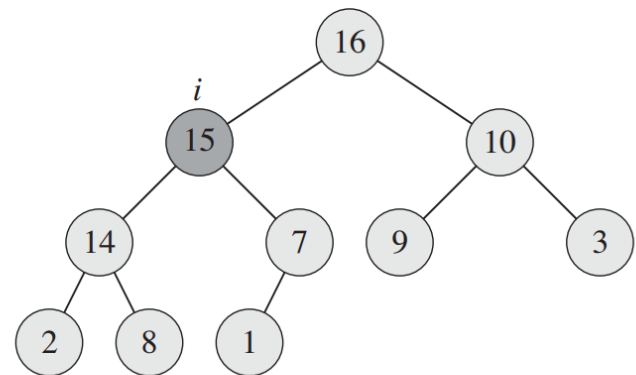
(a)



(b)



(c)

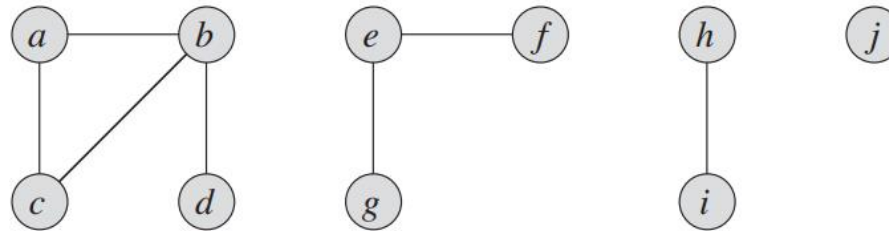


(d)

# Disjoint Sets

- A **disjoint-set data structure** maintains a collection  $\mathcal{S} = \{S_1, S_2, \dots, S_k\}$  of disjoint dynamic sets.
- Each set is identified by a **representative**, which is a some member of the set.
- $\text{Make-Set}(x)$  creates a new set whose only member is  $x$ .
- $\text{Union}(x, y)$  unites the sets that contain  $x$  and  $y$  into a new set that is the union of these two sets.
- $\text{Find-Set}(x)$  returns a pointer to the representative of the set containing  $x$ .

# Connected Components of an Undirected Graph



(a)

| Edge processed | Collection of disjoint sets |       |     |     |         |     |     |       |     |     |
|----------------|-----------------------------|-------|-----|-----|---------|-----|-----|-------|-----|-----|
| initial sets   | {a}                         | {b}   | {c} | {d} | {e}     | {f} | {g} | {h}   | {i} | {j} |
| (b,d)          | {a}                         | {b,d} | {c} |     | {e}     | {f} | {g} | {h}   | {i} | {j} |
| (e,g)          | {a}                         | {b,d} | {c} |     | {e,g}   | {f} |     | {h}   | {i} | {j} |
| (a,c)          | {a,c}                       | {b,d} |     |     | {e,g}   | {f} |     | {h}   | {i} | {j} |
| (h,i)          | {a,c}                       | {b,d} |     |     | {e,g}   | {f} |     | {h,i} |     | {j} |
| (a,b)          | {a,b,c,d}                   |       |     |     | {e,g}   | {f} |     | {h,i} |     | {j} |
| (e,f)          | {a,b,c,d}                   |       |     |     | {e,f,g} |     |     | {h,i} |     | {j} |
| (b,c)          | {a,b,c,d}                   |       |     |     | {e,f,g} |     |     | {h,i} |     | {j} |

(b)

# Connected Components of an Undirected Graph

CONNECTED-COMPONENTS( $G$ )

```
1  for each vertex  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for each edge  $(u, v) \in G.E$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )
```

SAME-COMPONENT( $u, v$ )

```
1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE
```

# Acknowledgements & References

- T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein, Introduction to Algorithms, 3<sup>rd</sup> edition, The MIT Press, 2009.